Distributed DAG Task Scheduler: Specification (MS3)

This is the living specification, revised for MS3. The approved MS1 claims are quoted at the top of each section; the text under them is what the MS3 system guarantees, under which conditions, and how to falsify it. Every claim maps to tests in the test-to-property matrix. Changes since MS2 are listed in §9 and explained in the MS3 progress report. The wire format is in api.md and the event schema in evidence.md.

1. Overview

> We are building a fault-tolerant distributed task scheduler for a video-processing platform. Each uploaded video creates a directed acyclic graph (DAG) of tasks such as transcoding, thumbnail generation, subtitle generation, and publishing. Some tasks can run in parallel, while others must wait for their dependencies. The system consists of one scheduler and multiple worker nodes. The scheduler tracks dependencies and assigns ready tasks to available workers. Workers execute tasks and report completion. If a worker fails before finishing, the unfinished task can be reassigned. We are not building a full video-sharing platform, authentication, billing, or production-quality video-processing algorithms.

A client submits a job as a JSON manifest describing a DAG of tasks drawn from a fixed operation allowlist. The system runs every task once its parents have succeeded, on whichever worker asks for work, and reports the job's outputs when every task has succeeded.

Component	Runs	Does	Talks to
Scheduler	one Java 21 process	Validates jobs, tracks dependencies, hands out ready tasks in FIFO order, grants and expires **leases**, accepts or rejects results. All decisions are made under one mutex on in-memory state.	Answers clients and workers over HTTP/JSON; checks output files exist in the artifact store (HEAD). Never calls workers.
Workers	any number of Java processes, one task at a time	Poll for work, get a start acknowledgment, run the operation (fixture arithmetic, `ffprobe`, FFmpeg transcode/thumbnail), upload outputs, report. Renew their lease with a heartbeat every second while they hold a task.	Scheduler (claim, start, heartbeat, report); artifact store (read inputs, write outputs). No inbound port.
Artifact store	one Java process	Stores immutable files under content-verified keys. A key, once published, never changes.	Answers workers, clients and the scheduler.
Client	CLI or the Python harness	Uploads inputs, submits jobs, polls status, downloads outputs.	Scheduler and artifact store.

The video job is `inspect` → {`transcode720`, `transcode360`, `thumbnail`, `subtitles`} in parallel → `publish`. `publish` writes a JSON manifest of the produced files; `subtitles` copies a fixture file rather than transcribing speech.

Capacity. A scheduler run keeps its complete decision history in memory so that every run can be audited: about 10 KB of heap per completed task (measured: `benchmarks/idle_memory.py`). With the default 128 MiB heap, one run holds about 11,000 tasks (about 1,900 six-task jobs); beyond that the

scheduler runs out of memory and has to be restarted, which starts a new, empty run (§5). Idle polling costs nothing: an idle system's memory stays flat.

Not built (anti-features). Scheduler replication, failover or durable scheduler state; artifact-store replication or recovery; exactly-once *physical* execution or exactly-once external side effects (a task can physically run more than once; only one result is ever accepted); authentication or authorization; a web UI; priorities, cancellation, deadlines or retry limits; real speech-to-text or production media pipelines; defense against malicious workers.

2. System model

- **Network:** fair-loss links: messages can be lost, delayed, duplicated and reordered, but a message that is retried repeatedly is eventually delivered. Endpoints are honest. Every state-changing request carries a stable identifier (job ID, claim ID, or the attempt's identity), so retrying it has no extra effect.
- **Workers:** crash-recovery. A worker can crash, freeze, run slowly, or be cut off at any moment, and may come back. A restarted worker is a new session and cannot act on its old attempts.
- **Scheduler:** assumed to stay up for the life of a run, and it is the only timekeeper. A restart starts a new, empty run with a new run ID; messages addressed to an old run get `409 STALE_RUN`.
- **Artifact store:** assumed available and correct; stored objects never change.
- **Timing:** partially synchronous. Safety never depends on timing. Progress needs the scheduler's process to keep running and, for a worker to keep a task, its heartbeats to reach the scheduler at least once per lease period. All durations are measured on one process's monotonic clock; ordering across machines comes from IDs and scheduler acknowledgments, never from comparing clocks.
- **Leases:** every attempt is owned through a lease of `LEASE_TIMEOUT_MS` (default 5000 ms) on the scheduler's clock. Assignment grants it; the start acknowledgment and each heartbeat renew it. Workers heartbeat every `lease/5` (1 s, measured from the end of the previous heartbeat), so up to three consecutive lost heartbeats are always tolerated. When the deadline passes the attempt becomes EXPIRED, the task returns to the back of the ready queue, and its next attempt gets a higher attempt number. That number is the fencing token: every later message carrying the old one is rejected.

3. Safety

> A task will not be marked complete unless successful completion is reported, and a task will not run before its required dependencies finish. Retried tasks will not be recorded as multiple successful completions. These claims are violated if an unfinished task is marked complete, a task runs before its dependencies, or one logical task is recorded as completed multiple times.

Status: all three hold, under every fault in §5. They do not depend on timing: a wrong lease setting can cost progress or duplicate work, never safety.

S1. No completion without a reported success. A task becomes SUCCEEDED only when the scheduler accepts a SUCCESS report from the attempt that currently holds its unexpired lease, and every declared output is already published in the artifact store with the reported length, SHA-256 and media type, under that attempt's own key prefix. Nothing else completes a task: not a timeout, not a lease expiry, not silence, not a file that merely exists, and not a success report from an older or expired attempt (`409 LEASE_EXPIRED` or `409 CONFLICT`). *Violation in the log:* a `task_succeeded` with no matching SUCCESS report from the current owner, or a `task_succeeded` for an attempt that already had a `lease_expired`. *Falsifying experiment:* let a worker's lease expire while it keeps working (drop its heartbeats, freeze it, or partition it), let it finish and report SUCCESS, and check whether that late report is accepted.

S2. No task before its dependencies. A task is assigned only after every parent has an accepted success, and a worker begins executing only after the scheduler acknowledged its start. Its inputs are exactly the parents' accepted outputs. *Violation:* a `task_assigned` before a parent's `task_succeeded`, or a `worker_operation_started` that does not join to that attempt's `task_started` acknowledgment. *Falsifying experiment:* submit joins and chains under concurrent workers, crashes and message loss, and compare every assignment with its parents' successes in the history.

S3. No duplicate completion. Each logical task `(job_id, task_id)` has at most one accepted success, and at most one attempt owns it at any moment. A replayed report gets the stored receipt back (`completion_duplicate`); a report or start from a superseded, failed or expired attempt is rejected without changing state. A job completes exactly once, after all its tasks. *Violation:* two `task_succeeded` for one task, two concurrent owners, or a `job_completed` before all tasks succeeded. *Falsifying experiment:* duplicate every report, drop replies so workers resend, crash workers after their success was accepted, and resume frozen workers after their task was reassigned.

S3 is about **recorded** completions. Physical execution is at-least-once: an expired attempt may still be running somewhere (frozen or partitioned) and may finish. Its outputs land under its own attempt prefix, are never referenced, and its report is rejected.

These invariants are checked automatically on the full recorded history of every service-backed test (`check_history`), and offline on saved evidence (`python -m tests.harness.check_evidence <dir>`). We assume honest workers: the scheduler checks identity, state and output integrity, but does not recompute results.

4. Liveness

> If workers remain available, every ready task will eventually be assigned and either complete or be retried after failure. If a worker fails while executing a task, that unfinished task will eventually become available for reassignment. This claim is violated if an unfinished task remains permanently stuck.

Status: both hold. (MS2 left L2 broken on purpose; MS3's leases fix it.)

L1. Ready tasks make progress. While at least one worker is running and can reach the scheduler and artifact store, every READY task is eventually assigned (the ready queue is FIFO, so a task waits only for the tasks ahead of it), and every attempt ends: it succeeds, fails explicitly (a crash in the operation, or `OPERATION_TIMEOUT_MS`, default 30 s, elapsing), or its lease expires. A failed or expired

attempt puts the task back in the queue with a higher attempt number. A worker that keeps renewing keeps its task for as long as its operation runs: slow is not dead. *Violation:* a READY task never assigned while a worker polls, or an attempt that never ends. *Falsifying experiment:* run many concurrent jobs, slow tasks longer than several leases, and short partitions; check that every job finishes and no live worker loses its task.

L2. A failed worker's task is reassigned. If the worker holding a task crashes, freezes, or loses contact with the scheduler for longer than the lease, the task's lease expires `LEASE_TIMEOUT_MS` after the worker's last accepted renewal (measured on the scheduler's clock); the next expiry scan, at most `LEASE_SCAN_MS` later plus scheduling delay, returns the task to the ready queue, and it is assigned to the next worker that asks. The quantitative bound is P-REC (§6). *Violation:* (the original MS1 wording) a task permanently stuck with a dead owner, or, quantitatively, a task still owned by a dead worker more than `LEASE_TIMEOUT_MS` + 2 s after its last renewal while a worker is polling. *Falsifying experiment:* kill or freeze a worker at each stage of an attempt (after claim, during the operation, after uploading but before reporting) and watch for reassignment.

When the system may stop making progress (by design): while no worker is running or reachable; during a permanent partition between workers and the scheduler; while the scheduler is down (its restart loses all jobs, §5); and for a task whose operation always fails, which is retried forever because there is no retry limit (the job stays RUNNING, never FAILED). A link that loses four or more consecutive heartbeats can make a live worker lose its task; that costs repeated work, never a duplicate completion. A link that is not fair-loss, for example one that drops every report from a worker but delivers its heartbeats, keeps that worker's task renewed and unfinished for as long as it lasts; this is outside the network model.

5. Fault model

> The system will handle worker crashes, worker restarts, temporary worker unavailability, and delayed or lost communication. Byzantine or malicious worker behavior is outside the scope of the project. These failures will be tested by stopping workers and introducing communication problems during active jobs.

Fault	In scope?	Behavior	Tested by
Worker crash at any point (after claim, while running, after upload, after reporting)	Yes	Lease expires; task requeued and reassigned; orphan outputs ignored.	crash tests at every stage, chaos
Worker restart	Yes	A new session; the old attempt can no longer act; its task is recovered by its lease.	restart test
Worker freeze, slowness, long GC (temporary unavailability)	Yes	Shorter than the lease: no effect. Longer: task reassigned, and the frozen worker's late messages are rejected; it then serves new work.	pause, slow-task, partition tests
Lost, delayed, duplicated, reordered worker↔scheduler messages	Yes	Retries reuse identifiers and get cached receipts; a lost reply causes a resend, never a second execution of the same attempt or a second completion.	proxy tests, chaos

Fault	In scope?	Behavior	Tested by
Worker↔scheduler partition	Yes	Shorter than the lease: the attempt survives. Longer: treated as a crash; the worker stands down when the link heals.	partition tests
Client retries	Yes	Same job ID and identical manifest replays; a different manifest under the same ID is `409`.	integration tests
Scheduler crash or restart	**No** (safe, not tolerated)	A new, empty run; in-flight and queued jobs are lost and must be resubmitted; old-run messages get `409 STALE_RUN`.	run-change tests
Artifact store unavailable	Partly	Submissions get `503` and nothing is accepted; success reports cannot be verified and are retried. Store crash or data loss is not handled.	storage-unavailable test
Scheduler↔artifact-store or worker↔artifact-store partition	No	Assumed available. Retries absorb short outages.	—
Malicious or Byzantine workers (forged identities, wrong results), clock tampering	No	—	—

Faults are injected by the harness: SIGKILL and SIGSTOP/SIGCONT on worker processes (Docker kill/pause on Compose), worker stall points enabled only in tests, and a per-worker fault proxy that drops requests or replies, delays, duplicates, or partitions that worker's link to the scheduler. Every injected fault is logged next to the histories.

6. Performance

> We will measure job completion time, task throughput, scheduling latency, and recovery time after worker failures. Performance will be evaluated by submitting increasing numbers of concurrent DAG-based jobs under normal operation and while intentionally introducing worker failures.

Metrics (all on the scheduler's monotonic clock): job completion time (accepted → completed), throughput (accepted successes per second over a run), scheduling latency (ready → assigned), and recovery time (a killed worker's last accepted renewal → the task's assignment to a new worker), split into detection (→ lease expiry) and requeue (expiry → new assignment).

Workload and load generation. A closed-loop driver keeps C ∈ {1, 4, 16} six-task jobs in flight (100 ms fixture tasks, 4 warmup jobs, 24 measured jobs), on fresh Compose deployments with 1 CPU and 512 MiB per service. `make bench` runs the fault-free matrix over W ∈ {1, 2, 4} workers (MS2); `make bench-recovery` runs W = 4 in two modes, baseline and crash, where two busy workers are SIGKILLed during each measured phase and replaced.

Guarantee (P-REC). A crashed worker's task is never detected earlier than `LEASE_TIMEOUT_MS` after its last renewal, and, when an idle worker is polling, is reassigned within `LEASE_TIMEOUT_MS` + 2 s of it.

Under load (no idle worker), the recovered task additionally waits behind the tasks already queued, by FIFO. The bound is asserted in the crash oracle and the crash-at-each-stage tests.

Measured results (details in the progress report, §E):

- **Recovery.** With W = 4 and two busy workers killed in each of 8 runs, 13 tasks were orphaned and all 13 were recovered. Detection took 5.00–5.09 s. Recovery took 5.00–5.10 s when a worker was free, and once 10.3 s, when every worker was busy and the task waited its turn in the queue.
- **Cost to a job.** A job hit by a crash finished about one lease later than its peers (median 6.3 s against 1.0 s at C = 1). Other jobs were not slowed.
- **Throughput.** It is set by the number of workers: about 4.2 tasks/s per one-slot worker, reaching 16.7 tasks/s with 4 workers at C = 16 (MS2 matrix). On the noisier host of the MS3 runs, fault-free throughput at W = 4 was 4.4, 14.8 and 11.9 tasks/s at C = 1, 4 and 16. A same-day control of the MS2 code showed that the gap is the host, not leases.
- **Scheduling latency.** An idle worker receives a ready task in about 5 ms.
- **Memory.** An idle scheduler's heap stays flat; real work costs about 10 KB per completed task (§1, Capacity).

Throughput and job time are characterizations, not commitments: they depend on the host and on the synthetic workload. A deployment trades recovery time against false expiry by choosing `LEASE_TIMEOUT_MS`; recovery time is approximately the lease, plus up to one scan interval, one poll interval and one claim round trip.

Load parameters and the bottleneck. Load is set by C (jobs in flight), W (workers) and task durations. At the loads above, workers are the bottleneck: throughput grows with W, and the scheduler gives an idle worker a ready task in about 5 ms. The scheduler handles every request under one mutex, and its request rate grows with W even when there is no work (each idle worker claims every `POLL_INTERVAL_MS`, each busy worker renews every second), so a large enough fleet would saturate it; we have not measured where. There is no admission control: load above capacity waits in the FIFO ready queue, and job time grows with the queue, up to the memory capacity in §1.

7. Configuration that changes behavior

Setting	Where	Default	Effect
`LEASE_TIMEOUT_MS`	scheduler	5000	Lease length: recovery time vs. tolerance to slow links.
`LEASE_SCAN_MS`	scheduler	100	How often expired leases are reclaimed without waiting for a claim.
`HEARTBEAT_INTERVAL_MS`	worker	lease/5 from `/v1/health`	Renewal period. Must stay well below the lease.
`POLL_INTERVAL_MS`	worker	100	Idle worker's claim interval.
`OPERATION_TIMEOUT_MS`	worker	30000	A longer-running operation is failed explicitly and retried.

Setting	Where	Default	Effect
`ENABLE_TEST_HOOKS`	worker	false	Test-only stall points and forced failures. Never enable in a deployment.

8. Resources and replication

Everything runs from the repository with `make test` (Docker only). The smoke, functional and two crash tests also run on the course Hokea cluster (`deploy/hokea/README.md`). The reference host for the numbers in §6 is one Windows 11 machine (12 CPUs, 16 GB RAM) running Docker Desktop 27.1.1 with a 3 GB Linux VM, with every service capped at 1 CPU and 512 MiB. The workloads are committed: a five-task arithmetic DAG (`result=22`), a six-task benchmark DAG (`result=9`), and a byte-reproducible 10-second 1280×720 synthetic video with fixture subtitles.

9. Changes since MS2

- **L2 now holds.** Leases, worker heartbeats (`POST /v1/attempts/heartbeat`), scheduler-side expiry (before every command and every 100 ms), the EXPIRED attempt state, and `409 LEASE_EXPIRED` for every late message from an expired attempt. MS2's intentional failing test passes unchanged in substance.
- **S1 sharpened:** "reported" now means reported by the current, unexpired lease holder; late successes are rejected even if no other worker has taken the task yet.
- **S3 sharpened:** at-least-once physical execution and exactly-once recorded completion are now stated separately.
- **Fault model sharpened:** each fault is listed in or out of scope, with its tests. Communication faults and pauses are now tested, not just planned.
- **Performance:** recovery time is defined, measured and bounded.
- **Course cluster:** the Hokea adapter no longer rejects the course runner's Hokea build, and the crash tests pass on the cluster.